

PREVENCIÓN DE FRAUDE

Un enfoque de Machine Learning orientado a maximizar la ganancia del negocio



AGENDA

01 Metodología del proyecto

02 Análisis inicial y planteamiento del problema

03 Formulación matemática de la ganancia (J)

04 Exploración de datos: EDA raw y EDA train

05 Feature engineering

06 Selección y evaluación de modelos

07 Modelo final y simulación en producción

08 Conclusiones y referencias



FASES DEL PROYECTO

00

analisis_inicial

Lectura del enunciado y definición de la función de ganancia a optimizar.



01

EDA

Exploración de datos crudos y de entrenamiento; pruebas estadísticas y partición train/test.



02

feature_engineer

Construcción de variables, validación estadística y agregaciones temporales.



03

model_selection

Entrenamiento, evaluación y elección del modelo final; simulación en producción.



04

results

Consolidación de resultados, conclusiones y entrega final.



CONTEXTO DEL PROBLEMA

○ Objetivo

Predecir si una transacción es fraudulenta, maximizando la ganancia de la empresa (no solo la exactitud).

○ Regla de negocio

+25% de margen por transacción legítima aprobada; -100% del monto por fraude aprobado (FN).

○ Enfoque

Cost-Sensitive Learning (Elkan, 2001): un FN cuesta 4x más que un FP.

	Pred: Fraude	Pred: Legítima
Real: Fraude	TP Costo = 0	FN -100% monto
Real: Legítima	FP -25% monto	TN +25% monto



FORMULACIÓN MATEMÁTICA A OPTIMIZAR

$$J = -1.00 \cdot FN - 0.25 \cdot FP$$

Función objetivo del modelo — daño financiero causado por los errores (≤ 0 ; óptimo = 0)

PASO 1

Ganancia general

$$J_{\text{gen}} = 0.25 \cdot TN - 1.00 \cdot FN - 0.25 \cdot FP$$

PASO 2

Término constante

$$C = 0.25 \cdot N_{\text{legítimas}}$$

PASO 3

Función a optimizar

$$J = J_{\text{gen}} - C$$

PASO 4

Umbral óptimo teórico

$$p^* = 0.25 / 1.25 = 0.20$$

PASO 5

En la práctica

TH óptimo calibrado vía sweep sobre test



DISTRIBUCIÓN Y MAGNITUD DEL FRAUDE

○ Tamaño del dataset

150,000 transacciones, 19 columnas. Periodo 2020-03-08 a 2020-04-21 (44 días).

150,000

transacciones

○ Fraude diario

la tasa de fraude diaria fluctúa entre ~3% y ~6%, sin tendencia marcada de crecimiento en el periodo.

7,500

fraudes (5.0%)

○ Impacto económico

bajo la regla 25%/100% se construyó una tabla de ganancia/pérdida acumulada, asumiendo que el fraude confirmado es fraude no contenido (hipótesis de trabajo).

44

días de historia

○ Calidad de datos

variables candidatas a ID identificadas y depuradas; revisión de nulos, ceros y concentración en variables categóricas.



PARTICIÓN TRAIN/TEST: POR QUÉ TEMPORAL

PARTICIÓN TEMPORAL (44 DÍAS)

TRAIN

2020-03-08 → 2020-04-09

104,981 filas · 5.18% fraude

TEST

2020-04-10 → 2020-04-21

45,019 filas · 4.57% fraude

Split temporal (train_test_split_time, test_size≈0.311) con align_to_day=True — evita fraccionar un mismo día entre particiones.

○ Simula el escenario real de producción

en producción siempre se predicen transacciones futuras nunca vistas; un split aleatorio (shuffsplit) filtraría información del futuro hacia el pasado, algo que nunca ocurre en el mundo real.

○ Evita fuga de información temporal

las features de agregación (sumas/promedios móviles por ventana de hasta 10 días) dependen del orden cronológico; mezclar días de train y test rompería esa causalidad y sobreestimaría el desempeño.

○ align_to_day=True

garantiza que un día completo quede íntegro en un solo lado de la partición, evitando que transacciones de las mismas horas contaminen train y test simultáneamente.

○ Proporción realista

test_size≈0.311 (14 de 45 días) deja suficiente historia en train para las ventanas de agregación más largas, sin sacrificar el tamaño del conjunto de test.

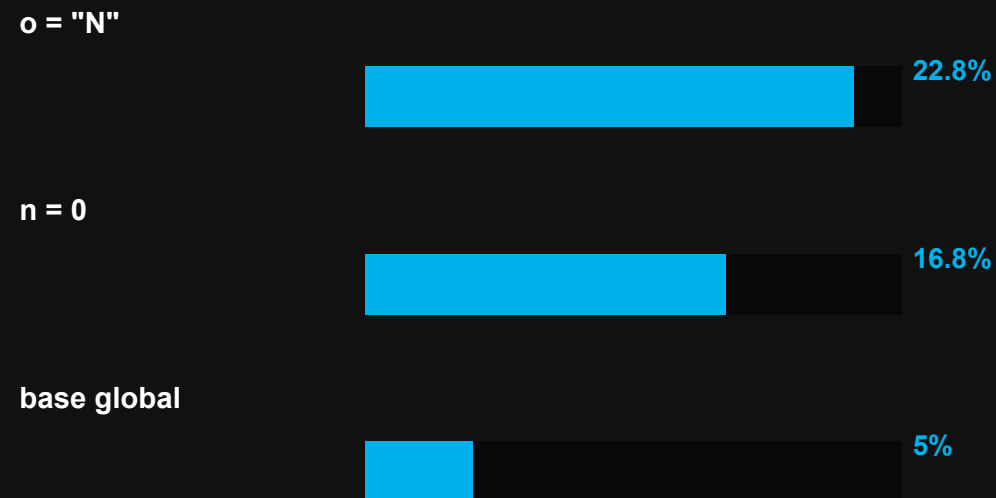


VARIABLES CANDIDATAS PARA EL MODELO

ASOCIACIÓN CATEGÓRICA (V DE CRAMÉR)



FRAUD RATE POR CATEGORÍA DE RIESGO



Variables continuas sin riesgo de multicolinealidad severa ($VIF \leq 1.61$): pueden incluirse todas. El EDA confirma señales categóricas y numéricas suficientemente fuertes para justificar el feature engineering dirigido.



CONSTRUCCIÓN DE NUEVAS VARIABLES

CALIDAD Y DISTRIBUCIÓN

- missing_flag
- zero_flag
- top_percentile_flag (q=0.99)
- log_feature (log1p)
- quantile_bins_dropna (5 bins)

ENCODING CATEGÓRICO

- frequency_encoding
- rare_grouping
- target_encoding_with_cross_val

TEMPORALES Y COMBINACIÓN

- hour / weekday
- is_weekend_flag
- hour_cyclic_sin_cos
- pca_components (d+m → 1 comp.)
- ratio (monto / l)

Enfoque config-driven (feature_engineer_config.yml + preprocessing.apply_transformations): cada técnica se aplica solo cuando es estructuralmente compatible y no degenerada (p.ej. no se agrega missing_flag si la variable no tiene nulos) — 61 features derivadas en total.



PRUEBAS ESTADÍSTICAS DE RELEVANCIA

VARIABLES CONTINUAS

- Mann-Whitney U
- Kolmogorov-Smirnov
- Punto-biserial
- ROC-AUC univariado
- Cohen's d
- Wald (logit)

VARIABLES CATEGÓRICAS

- Chi-cuadrado
- V de Cramér
- Information Value
- Weight of Evidence
- LR test (logit)

Implementado en FeatureStatTests (src/feature_engineer_utils/statistical_tests.py) — estandariza el cálculo de todas las pruebas por feature, cuantificando la relevancia de cada variable antes del modelado.



AGREGACIONES TEMPORALES (ROLLING WINDOWS)

VENTANAS TEMPORALES (agrupado por j, sobre monto)

1h

1d

3d

7d

10d

○ Ventanas evaluadas

1h, 1d, 3d, 7d y 10d, aplicadas sobre monto y agrupadas por la entidad categórica j.

○ Estadísticos calculados

suma, promedio, mediana, máximo y mínimo por ventana — 5 estadísticos x 5 ventanas.

○ Diseño causal

cada agregación usa únicamente información estrictamente anterior a la transacción actual, evitando data leakage hacia el futuro.

○ Relevancia

estas features capturan el comportamiento histórico reciente de una entidad (j), un patrón de alto valor predictivo en fraude transaccional.



MODELOS ENTRENADOS

MODELOS COMPARADOS

Logistic Regression

Random Forest

LightGBM

CatBoost

XGBoost

FEATURES DEL BASELINE (23) — mismo conjunto para los 5 modelos, vía `filter_config_by_features`

Variables crudas (passthrough)

a b c d e f h l m n p monto score

Encoding categórico

g_freq_encoded j_freq_encoded j_target_enc o_freq_encoded o_target_enc

Features engineered

f_is_zero score_qbin_dropna pca_dm_1 monto_per_l

Agregación temporal

monto_sum_15d

Persistencia: cada modelo se guarda en su propia subcarpeta — pipeline completo vía joblib, más el formato nativo del algoritmo (.txt LightGBM, .json XGBoost, .cbm CatBoost).



APLICACIÓN DE LA FUNCIÓN DE GANANCIA (J)

○ TH fijo vs. TH óptimo

cada modelo se evalúa con umbral fijo (0.5) y un umbral óptimo calibrado vía `sweep_business_profit`.

○ Métrica ancla

$J(\$) = -1.00 \cdot \sum \text{monto}(\text{FN}) - 0.25 \cdot \sum \text{monto}(\text{FP})$, calculada sobre el conjunto de test real.

○ Escenario base "bloquear todo"

$J = -\$428,970.04$ — referencia de costo de oportunidad de rechazar todas las transacciones.

GANANCIA POR CALIBRACIÓN DE UMBRAL

Logistic Regression



Random Forest



LightGBM



CatBoost



XGBoost



RESULTADOS Y MÉTRICAS (TH ÓPTIMO)

Modelo	AUC	PR-AUC	Precision (TH óptimo)	Recall (TH óptimo)	J óptimo (\$)	J block all (\$)	Mejora vs. TH fijo
LightGBM	0.8850	0.4354	0.3702	0.5005	-86,068	-428,970	+6.0%
CatBoost	0.8805	0.4368	0.3383	0.5418	-87,887	-428,970	+3.4%
XGBoost	0.8681	0.4203	0.3956	0.4495	-90,194	-428,970	+1.3%
Random Forest	0.8673	0.4049	0.3452	0.4985	-93,752	-428,970	+11.0%
Logistic Regression	0.7525	0.1907	0.3143	0.1788	-127,171	-428,970	+39.6%

Ranking por J óptimo: LightGBM ≈ CatBoost > XGBoost > Random Forest >> Logistic Regression · J block all = rechazar el 100% de las transacciones (referencia de piso, igual para los 5 modelos)



LIGHTGBM: MODELO ELEGIDO

○ Criterio de selección

mejor balance entre poder discriminativo (AUC 0.8850, PR-AUC 0.4354) y ganancia de negocio (J óptimo=-\$86,068).

○ Riesgo evitado

a diferencia de XGBoost, donde o_freq_encoded concentra 38.0% de la importancia (riesgo de concept drift), LightGBM reparte el poder predictivo.

○ Eficiencia operativa

diseñado para datasets grandes y baja latencia de entrenamiento/inferencia — relevante para scoring en tiempo real.

TOP 5 FEATURE IMPORTANCE (LIGHTGBM)

score



j_target_enc



l



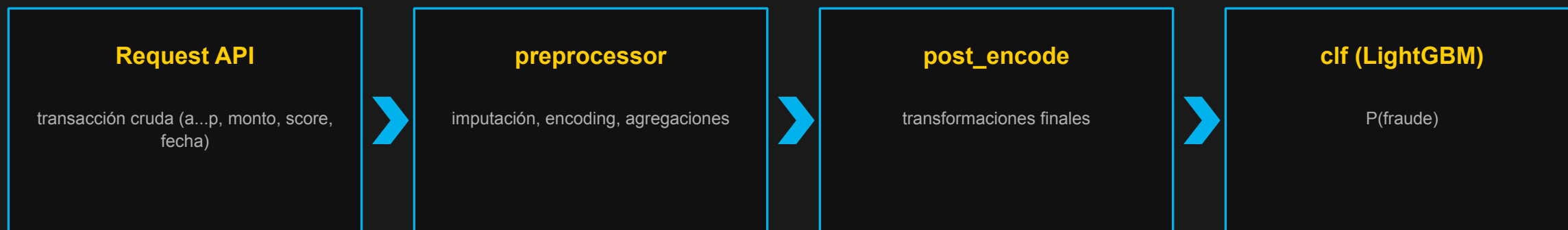
monto



c



SIMULACIÓN DEL MODELO EN PRODUCCIÓN



0.8850

AUC reproducido (idéntico al esperado)

0.0209

P(fraude) transacción → LEGÍTIMA (TH fijo y TH óptimo)

0.0062

P(fraude) con categorías j/g nunca vistas — no falla

El pipeline completo (preprocesamiento + modelo) es autocontenido y reproducible fuera del entorno de entrenamiento — condición necesaria para un despliegue confiable.



CONCLUSIONES GENERALES DEL PROYECTO

01

No es clasificación estándar

Optimización de ganancia bajo un costo asimétrico — un FN cuesta 4x más que un FP.

02

Señales identificadas en EDA

variables categóricas (j, o, n) y temporales suficientemente fuertes.

03

La calibración del umbral importa

tanto como el modelo — mejoras de hasta +39.6% en ganancia.

04

Modelo final: LightGBM

mejor ganancia de negocio, buen AUC y feature importance distribuida.

05

Simulación de producción exitosa

el pipeline serializado reproduce resultados y maneja categorías no vistas.

06

Próximos pasos

monitoreo de drift, reentrenamiento periódico y cost-sensitive learning en la función de pérdida.



MONITOREO DE DRIFT EN FEATURES

○ Por qué monitorear

un modelo en producción puede degradarse silenciosamente si las features de entrada cambian de distribución respecto al momento en que se entrenó — detectarlo a tiempo evita perder ganancia de negocio sin que nadie lo note.

○ Métricas utilizadas

KS (Kolmogorov-Smirnov) para las 10 variables continuas y PSI (Population Stability Index) para las 6 categóricas, comparando cada semana de train contra la semana 0 como referencia — más la tasa de nulos por separado, ya que el KS solo evalúa los valores no nulos.

○ Resultado principal

ninguna feature alcanza drift significativo ($KS > 0.20$ / $PSI > 0.25$); b, h y monto muestran drift leve hacia las semanas 3-4, coincidiendo con el aumento de la tasa de fraude semanal en ese tramo.

○ Hallazgo inesperado

b y c tienen 47.8% de nulos en la semana 0 y caen a ~1% desde la semana 1 — invisible para el KS (que solo mira valores no nulos), y más compatible con una inestabilidad puntual del feed de datos que con un drift natural.

MAYOR DRIFT OBSERVADO (KS / PSI)

h (KS)



b (KS)



monto (KS)



score (KS)



g (PSI)



CONSIDERACIONES ADICIONALES

¿Qué pasos puedo seguir para intentar asegurar que la performance del modelo en laboratorio será similar a la de producción?

Simulación del proceso productivo

Como se mencionó en slides anteriores, se realizó una simulación del proceso productivo, esto carga los artefactos y preprocesamientos ejecutados en el pipeline de entrenamiento, y asegura que se mantenga el comportamiento esperado para las transformaciones necesarias, este paso es crítico porque si no se garantiza la información de entrenamiento y test puede ser diferente al de producción.

Ambiente del modelo productivo

Considerar el ambiente que soportará el modelo productivo, es decir, en ambientes de desarrollo los procesos de entrenamiento y test se realizan por batch en ambientes controlados, pero en producción lo normal es que el modelo sea un endpoint que responda real time a una transacción, y genere un score de riesgo y su respectiva aprobación o no del fraude, sin embargo, la infraestructura productiva debe garantizar los tiempos de respuesta a nivel online (infraestructura elástica y escalable), además el código debe de estar lo suficientemente bien optimizado para que el modelo cumpla con los SLA permitidos, y que sus tiempos de respuesta sean los adecuados.

Monitoreo del modelo en producción

Realizar procesos de monitoreo, cuando un modelo está en servicio, muchas veces se pasa por alto monitorearlo en el tiempo, esto cae en el riesgo de haber drifts silenciosos que con el tiempo degradan el modelo, medir el drift del score y sus features de entrada es una buena práctica para detectar a tiempo si se verán afectadas las métricas del modelo y de negocio, se puede realizar un dashboard de monitoreo que revise 3 puntos fundamentales, respuesta del score del modelo, degradación de features de entrada, y métricas de ML y de negocio (esta última mide impacto en monto y en cantidad de transacciones), esto asegura que se pueda detectar a tiempo las degradaciones del modelo online y que se haga a tiempo su respectivo reentrenamiento.

MLOps end-to-end

Finalmente, en conclusión, garantizar procesos de MLOps el cual permite mantener el ciclo de vida end-to-end del modelo siempre y cuando sea posible.

Nota: estas respuestas son basadas en experiencia, sin embargo en los siguientes artículos se detallan procesos similares que sustentan estas respuestas:

Rules of Machine Learning: Best Practices for ML Engineering — developers.google.com/machine-learning/guides/rules-of-ml?hl=es-419

Monitor models for training-serving skew with Vertex AI — cloud.google.com/blog/topics/developers-practitioners/monitor-models-training-serving-skew-vertex-ai

Population Stability Index (PSI): What You Need To Know — arize.com/blog-course/population-stability-index-psi



CONSIDERACIONES ADICIONALES

CONSIDERACIONES ADICIONALES

Suponiendo que el desempeño predictivo en producción es muy diferente de lo que se esperaba, ¿cuáles crees que son las causas más probables?

Para analizar las causas más probables que puedan hacer que el desempeño no sea el esperado, puede haber varios puntos por revisar: ¿la infraestructura se mantiene sana y estable?, ¿no se rompió el endpoint de la API del modelo?, ¿se garantizó que las transformaciones utilizadas en Train se están aplicando de manera adecuada en Prod? En caso de que la infraestructura, el endpoint y el código sean estables, algunas causas probables pueden ser las siguientes:

Dependencia de reglas de negocio con el modelo

Hay procesos que aplican reglas de negocio mucho antes de que el modelo pueda operar. En estos casos, es posible que al modelo nunca le lleguen las suficientes transacciones y solo algunas pocas, muy diferentes a lo que se esperaba. Para estos casos lo ideal sería que el modelo fuera un sistema paralelo a las reglas y no en cascada; esto hace que el modelo pueda operar sobre toda la población y tomar una decisión sin depender de reglas externas.

Maduración de marca o target

Si el target a scorear tarda mucho en madurar, las métricas no podrán medirse de manera rápida, y se tendrá que esperar semanas o hasta meses para ver reflejado el verdadero performance de un modelo.

Cálculo distinto en la ETL

Si el modelo no depende de APIs que proveen las features, sino de una ETL, puede ocurrir que alguna feature con importancia alta se calcule diferente, cambiando la distribución de la misma.

Features esperadas que no llegan

Se esperaban valores en la "feature_1" pero solo llegan nulos o ceros. Esta causa es muy común: en Train las features pueden contener datos, pero a nivel productivo estos datos nunca llegan. Esto puede ocurrir en muchas features, y si están en el top de importancia del modelo, efectivamente su desempeño caerá estrepitosamente en un periodo corto de tiempo. Lo más recomendable es asegurar que en Prod todas las features tendrán los valores esperados para evitar esto.

Data Leakage en el entrenamiento

Garantizar que en el proceso de Train se haya evitado sí o sí el data leakage. Si esto no se evita, las métricas del proceso de entrenamiento fueron engañosas y se inflaron más de lo esperado, ocasionando que en Prod el modelo funcione peor.

Drift natural de las features

Los procesos son cambiantes y es posible que en el proceso de entrenamiento las features tuvieran una distribución muy diferente a la del momento en que se productivizó. Esto puede deberse a campañas de peak seasons o similares que alteran el comportamiento normal de las features y que el modelo no tenía en el proceso de Train.

Deprecación de features

En muchas ocasiones los modelos consumen APIs que proveen diferentes tipos de features. Si el owner de alguna de estas APIs considera darla de baja, al modelo no le llegarán esas features, ya que la API que las provee se rompe, ocasionando un problema similar al anterior.



CONSIDERACIONES ADICIONALES

¿Qué pasos se deben seguir para poner el nuevo modelo en producción?

○ Pipeline y artifacts configurados

Inicialmente se debe tener configurado el pipeline del modelo, los artifacts necesarios para el despliegue que garanticen el funcionamiento y las transformaciones de features.

○ Plan de rollout a producción

Tener el plan del rollout a producción, realizarlo en horarios pertinentes y que no afecten el freeze.

○ Configuraciones del sistema de rollout

Tener las configuraciones necesarias sobre el sistema o herramienta que permitirá hacer el rollout, en el caso de Mercado Libre (MPI, Scopes, entidades de features, ...).

○ Rollout controlado (Blue-Green / Canary)

Realizar el rollout de manera controlada, por defecto realizar Blue Green o Canary dependiendo del modelo en cuestión y flujo a scorear.

○ Dashboard de monitoreo en tiempo real

Este paso es crucial ya que después de que el modelo ya se encuentre online, es necesario realizar su monitoreo, en cuanto a tiempos de respuesta y errores, en caso tal de encontrar inconsistencias realizar el rollback adecuado o bajada del mismo.



REFERENCIAS BIBLIOGRÁFICAS

- ▶ The Foundations of Cost-Sensitive Learning — Charles Elkan (2001)
- ▶ The Missing Indicator Method: From Low to High Dimensions
- ▶ Impact of Sampling Techniques and Data Leakage on XGBoost Performance in Credit Card Fraud Detection
- ▶ Why do Tree-Based Models Still Outperform Deep Learning on Tabular Data?

